KIẾN TRÚC PHẦN MỀM (SEI & ISO 25010)

QUALITY ATTRIBUTES & ARCHITECTURAL TACTICS

Phân tích hệ thống đặc tính chất lượng, kịch bản đặc tả chuẩn SEI 6 thành phần và thực chứng trên Microservices Demo (Google Online Boutique)

Bộ môn / Trường
Khoa CNTT - ĐH Khoa học Tự nhiên (HCMUS)

Giáo trình tham chiếu
Software Architecture in Practice (Bass, Clements, Kazman)

Trọng tâm thực hành
Demo Scalability (K6 Load Test) & Security Audit

01 Tổng Quan Quality Attributes & Kịch Bản Chuẩn SEI

NỀN TẢNG KIẾN TRÚC

Bản chất cốt lõi: Tính năng chức năng (*Functionality*) không định hình nên cấu trúc kiến trúc; nếu chỉ cần chức năng, một khối mã nguồn duy nhất (Monolithic blob) là đủ. **Chính các thuộc tính chất lượng (Quality Attributes)** — hiệu năng, an ninh, độ sẵn sàng, khả năng mở rộng — mới quyết định việc chia tách module, lựa chọn pattern và hạ tầng triển khai.

📐 Vai Trò Trong Đánh Giá Kiến Trúc (ATAM)

- **Đặc tả phi chức năng định lượng:** Tránh các mô tả mơ hồ như "hệ thống phải nhanh" hoặc "hệ thống phải bảo mật cao".
- **Cơ sở lựa chọn Tactics:** Mỗi đặc tính chất lượng được hiện thực hóa thông qua các quyết định kiến trúc cụ thể (Tactics).
- **Phát hiện Trade-offs:** Không thể tối ưu đồng thời mọi đặc tính; kiến trúc là nghệ thuật cân bằng sự đánh đổi.

🎯 Khung Đo Lường & Chuẩn Đánh Giá

- **Mô hình SEI / ATAM:** Phân tích phản hồi của kiến trúc trước kích thích từ môi trường.
- **Mô hình ISO/IEC 25010:** Hệ thống hóa 8 đặc tính chất lượng của sản phẩm phần mềm.
- **Kịch bản đặc tả (Scenario):** Công cụ giao tiếp duy nhất giữa các Stakeholder và Kiến trúc sư.

CẤU TRÚC 6 THÀNH PHẦN CHUẨN SEI ĐỂ ĐẶC TẢ QUALITY ATTRIBUTE SCENARIO

1. STIMULUS SOURCE	2. STIMULUS	3. ARTIFACT	4. ENVIRONMENT	5. RESPONSE	6. RESPONSE MEASURE
Tác nhân sinh kích thích (User, Attacker, Timer, Failure).	Điều kiện hoặc sự kiện tác động vào hệ thống (Request tăng vọt, Node crash).	Bộ phận kiến trúc chịu tác động (Frontend, Database, Message Queue).	Trạng thái lúc xảy ra kích thích (Bình thường, Quá tải, Đang triển khai).	Hành vi phản hồi của kiến trúc (Scale instance, Cô lập lỗi, Chặn token).	Chỉ số đo lường kết quả (p95 < 1.5s, Uptime 99.9%, 0 data loss).

02 Scalability (Khả Năng Mở Rộng) & Demo Thực Tế

★ TRỌNG TÂM DEMO

Định nghĩa: Khả năng duy trì hoặc tăng năng lực phục vụ (throughput, latency ổn định) khi tải gia tăng bằng cách bổ sung tài nguyên phần cứng (CPU, Memory, Node) mà không cần cấu trúc lại mã nguồn.

⚙️ Architectural Tactics Đạt Scalability

- **Scale-out (Horizontal Scaling):** Nhân bản microservice stateless qua Load Balancer (Nginx, Kube-proxy).
- **State Decoupling:** Đưa session/state ra Distributed Cache (Redis) để backend hoàn toàn không trạng thái.
- **Database Read-Replication:** Tách tải truy vấn Đọc (CQRS) giảm áp lực cho master database.
- **Asynchronous Decoupling:** Hàng đợi Message Queue làm buffer đệm đỉnh tải.

📊 Chỉ Số Đo Lường & Công Cụ Test

- **Throughput (RPS):** Lượng request hoàn thành hợp lệ trên 1 đơn vị thời gian.
- **Tail Latency p(95) / p(99):** Độ trễ của nhóm request chậm nhất khi chịu tải cao.
- **Error Rate under Stress:** Tỉ lệ request bị timeout hoặc HTTP 5xx.
- **Công cụ kiểm thử:** `k6` (Constant Arrival Rate), `Locust`, `JMeter`.

🚀 DỮ LIỆU THỰC CHỨNG TRÊN GOOGLE ONLINE BOUTIQUE (350 req/s cố định trong 20s = 7,000 reqs)

Cấu hình thử nghiệm	Tổng Request xử lý	Thành công (HTTP 200)	Thất bại (Timeout > 1.5s)	Độ trễ p(95)	Kết luận kiến trúc
1 Replica Frontend (0.25 CPU)	5,317 reqs	**228 reqs (4.28%)**	**5,089 reqs (95.71%)**	**> 1.50s (Nghẽn)**	CPU kiệt quệ, vỡ hàng đợi TCP socket
3 Replicas Frontend (Nginx Balancer)	6,506 reqs	**6,494 reqs (99.81%)**	**12 reqs (0.18%)**	**1.33s (< 1.5s)**	**Tải phân bổ đều 1/3, SLA đạt chuẩn 99.8%**

03 Security (An Ninh & Bảo Mật) & Kết Quả Audit

★ TRỌNG TÂM DEMO

> **Định nghĩa:** Khả năng bảo vệ tài sản thông tin chống lại truy cập, sửa đổi, phá hoại trái phép (tam giác CIA: Confidentiality, Integrity, Availability), đồng thời đảm bảo quyền truy cập cho người dùng hợp lệ.

Architectural Tactics (SEI Standard)

- **Detect Attacks:** Kiểm tra tính toàn vẹn thông điệp, IDS/WAF phát hiện brute-force.
- **Resist Attacks:** Xác thực (JWT/OAuth2), Phân quyền (RBAC/ABAC), Mã hóa at-rest (AES-256) và in-transit (mTLS).
- **React to Attacks:** Thu hồi phiên (Session revocation), Khóa tài khoản, Rate limiting.
- **Recover:** Khôi phục dữ liệu từ audit trail, đảm bảo tính chống chối bỏ (Nonrepudiation).

Phương Pháp & Công Cụ Kiểm Thử

- **SAST (Static Analysis):** Rà soát lỗ hổng mã nguồn (Semgrep, SonarQube).
- **SCA (Software Composition Analysis):** Quét thư viện phụ thuộc (Trivy, Snyk).
- **DAST (Dynamic Analysis):** Kiểm thử hộp đen lúc ứng dụng chạy (OWASP ZAP).
- **Metrics:** Số lượng CVE theo chuẩn CVSS (Critical/High), MTTD, MTTP.

⚠ CÁC LỖ HỔNG AUDIT TRỰC TIẾP TRÊN MÃ NGUỒN GOOGLE ONLINE BOUTIQUE

Mã lỗi & Chuẩn CWE	Vị trí mã nguồn lỗi	Rủi ro bảo mật thực tế	Giải pháp khắc phục kiến trúc
CWE-614 / CWE-1004	`frontend/middleware.go:97`	Cookie session thiếu `HttpOnly`, `Secure`, `SameSite` (XSS cướp session)	Bắt buộc gán đủ flags bảo vệ cookie trong middleware
CWE-352 (No CSRF)	`frontend/handlers.go:211`	Endpoint `POST /cart` và checkout không có Anti-CSRF token	Tích hợp CSRF token middleware (như `gorilla/csrf`)
CWE-601 (Open Redirect)	`frontend/handlers.go:521`	Redirect trực tiếp qua Header `Referer` không kiểm duyệt whitelist domain	Validate whitelist domain trước khi gán header `Location`
CWE-319 / PCI-DSS	`frontend/main.go:230`	gRPC nội bộ chạy `insecure.NewCredentials()`, thẻ tín dụng truyền plaintext	Thiết lập Zero-Trust mTLS giữa các microservices

04 Availability (Độ Sẵn Sàng & Chống Chịu Sự Cố)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Tỉ lệ thời gian hệ thống vận hành đúng chức năng và sẵn sàng phục vụ khi được yêu cầu. Phân biệt: *Fault* (khiếm khuyết/sự cố nội bộ) nếu không được cô lập và xử lý sẽ chuyển hóa thành *Failure* (sụp đổ toàn hệ thống).

🔍 Phát Hiện Lỗi (Detect Faults)

- **Ping / Echo:** Kiểm tra liveness định kỳ qua giao thức mạng.
- **Heartbeat:** Node chủ động gửi thông điệp sống còn về orchestrator.
- **Kubernetes Probes:** Liveness probe (tự kill pod treo), Readiness probe (dừng cấp tải pod chưa sẵn sàng).
- **Timeout Detection:** Ngắt kết nối sớm khi dịch vụ phụ thuộc không trả lời.

🔄 Phục Hồi Lỗi (Recover)

- **Active Redundancy (Hot Spare):** Nhiều node chạy đồng thời, cùng nhận tải (Zero downtime failover).
- **Passive Redundancy (Warm/Cold):** Node dự phòng khởi động thay thế khi node chính sập.
- **Exponential Backoff Retry:** Thử lại có độ trễ lũy tiến tránh làm chết thêm downstream.
- **Graceful Degradation:** Trả về dữ liệu fallback khi service con chết (ví dụ: giỏ hàng trống).

🛑 Ngăn Ngừa Lỗi (Prevent)

- **Circuit Breaker:** Ngắt ngay lập tức các request tới service đang bị lỗi, chống domino failure.
- **Transactions & Rollback:** Đảm bảo tính toàn vẹn dữ liệu ACID hoặc Sagas.
- **Removal from Service:** Tự động loại bỏ node lỗi ra khỏi cụm trước khi người dùng gặp lỗi.
- **Rate Limiting / Throttling:** Giữ tải hệ thống dưới ngưỡng sập.

SCENARIO MẪU: THANH TOÁN KHI RECOMMENDATION SERVICE SẬP

STIMULUS SOURCE	STIMULUS	ARTIFACT	ENVIRONMENT	RESPONSE	RESPONSE MEASURE
Recommendation pod bị crash do Out Of Memory (OOM).	Hàng nghìn request thanh toán giỏ hàng gửi đến Frontend.	Frontend và Cart Service.	Vận hành bình thường trong giờ cao điểm.	Circuit breaker ngắt gọi Ad/Recommend, fallback giao diện tối giản.	Luồng thanh toán hoàn tất 100%, uptime duy trì 99.99%.

05 Performance (Hiệu Năng & Đáp Ứng Thời Gian)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Khả năng của hệ thống đáp ứng các ràng buộc về thời gian (timing constraints) khi xử lý các kích thích đầu vào. Phân biệt rõ: **Performance** là độ trễ/tốc độ ở một mức tải nhất định; **Scalability** là khả năng duy trì Performance khi tải gia tăng liên tục.

Tactic: Kiểm Soát Nhu Cầu (Control Demand)

- **Manage Sampling Rate:** Giảm tần suất gửi sự kiện sensor/telemetry không cần thiết.
- **Event Prioritization:** Sử dụng hàng đợi ưu tiên (Priority Queue) để tác vụ khẩn cấp (thanh toán) được xử lý trước tác vụ phụ (gửi email).
- **Reduce Communication Overhead:** Thay thế chuẩn JSON/HTTP/1.1 cồng kềnh bằng giao thức nhị phân gRPC/Protobuf trên HTTP/2.
- **Bound Queue Sizes:** Giới hạn độ dài hàng đợi tránh làm đầy bộ nhớ (OOM) khi tắc nghẽn.

Tactic: Quản Lý Tài Nguyên (Manage Resources)

- **Non-blocking Asynchronous I/O:** Dùng Event loop, Goroutines, Reactive streams tránh nghẽn thread.
- **Multi-layer Caching:** Cache tại RAM (In-memory), Cache phân tán (Redis), Cache biên (CDN cho ảnh/tệp tĩnh).
- **Data Replication / Read Replicas:** Nhân bản dữ liệu đọc gần máy chủ xử lý logic.
- **Connection Pooling:** Tái sử dụng TCP socket và Database Connection (keep-alive) tránh bắt tay 3 bước lặp lại.

Các Chỉ Số Đo Lường Cốt Lõi Cần Nắm Chắc Khi Trả Lời Vấn Đáp

1. Latency vs. Throughput:
• *Latency:* Thời gian phản hồi 1 request (ms).
• *Throughput:* Số lượng tác vụ hoàn thành / giây (RPS, TPS).

2. Percentiles (Tail Latency):
• *Average:* Thường bị bóp méo bởi số ngoại lai.
• *p(95), p(99):* Chuẩn công nghiệp phản ánh trải nghiệm của 1-5% người dùng chậm nhất.

3. Resource Saturation:
• % CPU, RAM, Disk I/O, Network bandwidth.
• Định luật Amdahl: Giới hạn tăng tốc khi song song hóa logic tuần tự.

06 Modifiability (Khả Năng Sửa Đổi & Bảo Trì)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Chi phí, thời gian và mức độ rủi ro khi thực hiện các thay đổi đối với hệ thống phần mềm (sửa lỗi, nâng cấp công nghệ, bổ sung chức năng mới) mà không làm ảnh hưởng tiêu cực đến các chức năng hiện tại.

🧩 Tactics Giảm Chi Phí Thay Đổi (SEI)

- **Reduce Size of Module:** Chia nhỏ module theo nguyên lý Đơn trách nhiệm (Single Responsibility Principle - SRP).
- **Increase Cohesion:** Đảm bảo các thành phần bên trong module phục vụ cùng một mục đích nghiệp vụ thống nhất.
- **Reduce Coupling:** • Đóng gói qua Interface và Dependency Injection (Inversion of Control).
  - Dùng cấu trúc trung gian (Intermediary / Broker / API Gateway).
  - Kiến trúc Event-Driven (Publish-Subscribe) loại bỏ phụ thuộc gọi hàm trực tiếp.
- **Defer Binding Time:** Dời thời điểm liên kết mã nguồn sang lúc triển khai: file cấu hình `.env`, Dynamic Loading, Feature Flags.

📉 Hiệu Ứng Gợn Sóng (Ripple Effect) & Đo Lường

- **Ripple Effect:** Khả năng một thay đổi tại một module dẫn đến chuỗi thay đổi bắt buộc ở các module khác. Kiến trúc tốt phải chặn đứng hiệu ứng này tại ranh giới module.
- **Chỉ số đo lường (Metrics):**
  - *Number of affected modules:* Số module phải sửa khi có 1 yêu cầu nghiệp vụ mới.
  - *Cyclomatic Complexity (CC):* Độ phức tạp luồng điều khiển của hàm/class.
  - *Coupling Between Objects (CBO):* Mức độ liên kết giữa các class.
  - *Cost in Person-hours:* Số giờ công thực tế để hoàn thành và test một thay đổi.

SCENARIO MẪU: TÍCH HỢP CỔNG THANH TOÁN MỚI (MOMO/ZALOPAY)

STIMULUS SOURCE	STIMULUS	ARTIFACT	ENVIRONMENT	RESPONSE	RESPONSE MEASURE
Nhà phát triển / Đội ngũ tích hợp đối tác.	Yêu cầu bổ sung thêm phương thức thanh toán ví điện tử mới.	Payment Service và Checkout Service.	Giai đoạn phát triển mã nguồn (Design time).	Chỉ cần implement `PaymentGateway` Interface mới, cấu hình qua file config.	Không sửa đổi code của Checkout service, hoàn thành và kiểm thử dưới 2 ngày công.

07 Testability (Khả Năng Kiểm Thử)

LÝ THUYẾT VẤN ĐÁP

Định nghĩa: Mức độ dễ dàng mà phần mềm cho phép người kiểm thử thiết kế kịch bản và thực thi nhằm phát hiện lỗi. Một kiến trúc có testability cao là kiến trúc *"dễ dàng bộc lộ khuyết tật" (gives up its faults easily)* thông qua kiểm tra tự động.

💉 Tactics Nâng Cao Testability (SEI)

- **Control & Observe System State (Kiểm soát và Quan sát):** • Cung cấp giao diện kiểm thử chuyên biệt (Specialized Test Interfaces / Debug endpoints).
  • Endpoint reset state database về trạng thái mẫu ban đầu (Fixtures).
  • Health check endpoints chi tiết tình trạng kết nối Redis/DB (liveness, readiness).
- **Abstract Data Sources (Trừu tượng hóa nguồn dữ liệu):** • Tách biệt Data Access Layer (Repository Pattern) cho phép dễ dàng thay thế bằng Mock/Stub trong Unit Test mà không cần DB thật.
- **Limit Architectural Complexity (Hạn chế độ phức tạp):** • Tránh phụ thuộc vòng (Circular dependencies).
  • Hạn chế tính bất định (Non-determinism) trong xử lý đồng thời đa luồng.

📈 Đo Lường & Kim Tự Tháp Kiểm Thử

- **Kim tự tháp kiểm thử (Test Pyramid):** • *Unit Test (70%):* Nhanh, cô lập, rẻ.
  • *Integration Test (20%):* Kiểm tra giao tiếp giữa các microservices (gRPC contracts).
  • *E2E Test (10%):* Chậm, tốn kém, kiểm thử toàn luồng người dùng.
- **Chỉ số đánh giá (Metrics):**
  • *Code Coverage:* Tỉ lệ dòng lệnh (Statement) và nhánh (Branch) được test bao phủ.
  • *Test Suite Execution Time:* Thời gian hoàn tất toàn bộ test suite trên CI/CD pipeline (phải dưới 10 phút để phản hồi nhanh).
  • *Fault Isolation:* Mức độ dễ dàng định vị lỗi khi một test case bị fail.

SCENARIO MẪU: KIỂM THỬ ĐỘC LẬP CART SERVICE BẰNG MOCK REDIS

STIMULUS SOURCE	STIMULUS	ARTIFACT	ENVIRONMENT	RESPONSE	RESPONSE MEASURE
Hệ thống CI/CD pipeline khi có Commit / Pull Request mới.	Thực thi bộ Unit & Integration Test tự động.	Cart Service và Mock `ICartStore`.	Môi trường container CI cô lập (không có Redis cluster thật).	Mock store tiêm vào service thành công, giả lập đầy đủ các tình huống lỗi mạng.	Branch coverage đạt > 85%, toàn bộ test suite hoàn tất trong dưới 60 giây.

08 Deployability & Usability

LÝ THUYẾT VẤN ĐÁP

🚀 1. Deployability (Khả Năng Triển Khai)

Khả năng đưa bản cập nhật phần mềm vào môi trường vận hành thực tế trong thời gian dự đoán được, với công sức tối thiểu và không làm gián đoạn người dùng.

- **Tactics Triển Khai Không Gián Đoạn (Zero-Downtime):**
  • *Containerization:* Đóng gói đồng nhất qua Docker, loại bỏ lỗi "chạy được trên máy tôi".
  • *Blue-Green Deployment:* Duy trì 2 môi trường song song; chuyển hướng traffic router tức thời.
  • *Canary Release:* Đẩy phiên bản mới cho 5% người dùng thử nghiệm trước khi mở rộng 100%.
  • *Automated Rollback:* Tự động revert bản cũ nếu tỉ lệ lỗi HTTP 5xx tăng vọt.
- **Chỉ số DORA:** Deployment Frequency (tần suất deploy), Lead Time for Changes (thời gian đưa code lên prod), Change Failure Rate.

👤 2. Usability (Khả Năng Sử Dụng)

Mức độ dễ dàng, thuận tiện và trực quan để người dùng hoàn thành mục tiêu tác vụ, cùng các cơ chế trợ giúp người dùng của hệ thống.

- **Tactics Cốt Lõi Theo Giáo Trình SEI:**
  • *Maintain Task Model:* Hệ thống hiểu ngữ cảnh người dùng đang làm gì để chủ động hỗ trợ (gợi ý tìm kiếm Type-ahead, tự động điền form, tự sửa lỗi chính tả).
  • *Maintain User Model:* Lưu trữ thói quen, quyền hạn, lịch sử và tùy biến giao diện cá nhân hóa theo từng người dùng.
  • *Maintain System Model:* Cung cấp phản hồi trạng thái minh bạch cho người dùng (Thanh tiến trình upload, thông báo lỗi dễ hiểu, khả năng Undo/Redo).
- **Chỉ số đo lường:** Task completion time (thời gian làm xong tác vụ), Error rate của người dùng, Điểm khảo sát chuẩn SUS (System Usability Scale).

💡 Tác Động Của Kiến Trúc Đến Usability (Không Chỉ Là Thiết Kế UI/UX)

Usability không đơn thuần là thẩm mỹ đồ họa CSS mà bắt nguồn từ các quyết định kiến trúc nền tảng: Kiến trúc phải cung cấp tính năng **Hủy tác vụ (Cancel)** khi request đang xử lý dở dang; tính năng **Hoàn tác (Undo)** đòi hỏi kiến trúc phải lưu trữ Command Pattern hoặc Memento Pattern; và tính năng phản hồi trạng thái đòi hỏi kết nối thời gian thực **Websocket / Server-Sent Events** từ backend.

09 Ma Trận Đánh Đổi Kiến Trúc (Architectural Trade-offs)

TỔNG HỢP & VẤN ĐÁP

> **Chân lý kiến trúc phần mềm:** Không tồn tại một kiến trúc "hoàn hảo" cho mọi thuộc tính chất lượng. Việc tối ưu hóa một thuộc tính chất lượng này hầu như luôn dẫn đến sự suy giảm của một hoặc nhiều thuộc tính chất lượng khác.

Cặp Đánh Đổi Cốt Lõi	Bản Chất Mâu Thuẫn Kiến Trúc	Chiến Lược Cân Bằng Thực Tế (Tactics)
Scalability vs. Consistency (Định lý CAP)	Khi scale-out dữ liệu sang hàng chục node phân tán để chịu tải cao, việc bắt buộc tất cả node phải đồng bộ dữ liệu tức thời (Strong Consistency) sẽ làm nghẽn mạng và sụt giảm nghiêm trọng Availability.	Chấp nhận mô hình **Eventual Consistency** (Nhất quán cuối cùng). Áp dụng *Saga Pattern* hoặc Outbox Pattern với Message Queue thay cho Distributed Two-Phase Commit (2PC).
Security vs. Performance	Việc áp dụng mã hóa toàn diện (mTLS), phân tích chữ ký số, giải mã payload và kiểm tra token trên từng request sẽ tiêu tốn chu kỳ CPU và tăng độ trễ mạng (Network Latency).	Dùng Hardware Crypto Acceleration (TLS offloading tại API Gateway), Cache token xác thực phía Gateway, chỉ áp dụng mTLS nội bộ cho luồng dữ liệu tài chính cực kỳ nhạy cảm.
Performance vs. Modifiability	Tối ưu hiệu năng cực đoan bằng cách gộp chung module, viết code bậc thấp, chia sẻ trực tiếp bộ nhớ hoặc nhúng cache đa tầng khiến code phức tạp, gắn kết chặt chẽ và rất khó sửa đổi.	Áp dụng Clean Architecture / Microservices phân tách ranh giới rõ ràng. Chỉ tối ưu cục bộ tại đúng **điểm nghẽn cổ chai (Bottleneck)** sau khi có kết quả Profiling thực tế.
Availability vs. Financial Cost	Để đạt mức sẵn sàng "năm số 9" (99.999% - chỉ được phép downtime 5 phút/năm), hệ thống bắt buộc phải nhân bản toàn bộ hạ tầng Active-Active trên nhiều Data Center khác nhau, chi phí đội lên gấp nhiều lần.	Xác định rõ SLA/SLO theo giá trị nghiệp vụ: Chấp nhận 99.9% ("ba số 9") cho các dịch vụ phụ (gợi ý sản phẩm, giỏ hàng), chỉ đầu tư 99.99% cho luồng thanh toán cốt lõi.

🎓 **THÔNG ĐIỆP KẾT LUẬN VẤN ĐÁP:**

"Mục tiêu của người kiến trúc sư phần mềm không phải là xây dựng một hệ thống tối đa hóa tất cả các chỉ số chất lượng, mà là **nhận diện chính xác mục tiêu nghiệp vụ tối thượng của hệ thống**, từ đó đưa ra các quyết định đánh đổi kiến trúc (Trade-offs) có cơ sở khoa học và định lượng được bằng kịch bản kiểm thử."